AI FLUENCY HACKATHON — PLANNING CONVERSATION

Understanding & Building an Agent Harness

A working session on what a harness is and how to build one — scoped toward a design-handoff & downstream-communications agent.

Participant: Brian Studwell, Product Design **Date:** 8 July 2026 **Assistant:** Claude Code

BRIAN

> I'm participating in a company-wide hackathon to up-skill our teams and promote AI fluency. I'm a member of our product design team who uses Claude Code every day for work. We're currently in the weeks leading up to the actual 2-day build. During the build days we'll be asked to build an agent that can perform some kind of work on our behalfs. My project is going to focus on design handoff and subsequent communications.
>
> One task every team, mine included, is being asked to do is to build out a harness for their agent. Not being very familiar with harnesses, I'd like your help in this effort. Bring me up to speed on what a harness is. I don't have to have a PhD, just a solid understanding. Then help me identify the steps I need to take to build a harness. Ask me clarifying questions to flesh out the task.

CLAUDE

Great project to pick — design handoff is a genuinely good agent use case, and understanding harnesses will make you dangerous (in a good way) at this hackathon. Let me get you oriented, then we'll scope your build.

What a harness actually is

A language model on its own is just a text predictor. You give it text, it gives you text back. That's it. It can't open a file, send an email, remember what happened yesterday, or take a second step based on the result of a first step.

A harness is all the scaffolding around the model that turns "text in, text out" into an agent that can actually do work. It's the machinery that:

- **Feeds the model the right context** (instructions, relevant docs, the current state of things)
- **Gives it tools** and runs them when the model asks (read a file, call an API, send a message)
- **Loops** — takes the model's output, executes any actions, feeds the results back in, and repeats until the job is done
- **Keeps it on the rails** — permissions, guardrails, what it's allowed to touch, when to stop and ask a human

A useful analogy: the model is an engine. The harness is the rest of the car — the transmission, steering, dashboard, brakes, and the seatbelt. A brilliant engine sitting on the floor doesn't take you anywhere.

You're already using a harness every day: Claude Code is one. The model doesn't inherently know how to read your files or run bash — Claude Code (the harness) gives it those tools, manages what context the model sees, runs the tools when the model requests them, loops until your task is done, and asks your permission before risky actions. Your hackathon task is to build a purpose-built version of that same idea, aimed at design handoff.

The anatomy of a harness (the 6 pieces you'll design)

1. **The loop** — the core cycle: model thinks → requests an action → harness runs it → result goes back to model → repeat until done. This is the beating heart.
2. **Instructions / system prompt** — the agent's job description, personality, rules, and definition of "done."
3. **Context management** — what information the agent gets and when. Too little and it's clueless; too much and it drowns.
4. **Tools** — the concrete capabilities. Each is a function the model can invoke: read a Figma file, draft an email, post to a channel, look up a component spec.
5. **Guardrails & control** — permissions, approval gates, what it can't do, when to hand back to a human. Critical for anything that talks to the outside world (like sending comms).
6. **Evaluation** — how you know it's doing a good job. Test cases, examples of good vs. bad output, a way to measure it.

The steps to build one

1. **Define the job precisely** — one clear task the agent owns end-to-end.
2. **Map the workflow you do today** — the manual steps, the inputs you look at, the outputs you produce, the decisions you make. The agent is automating *this*.
3. **Inventory the tools it needs** — for each manual step, what capability does the agent need?
4. **Write the instructions** — role, process, rules, output format, when to stop and ask you.
5. **Decide context strategy** — what you'll feed it up front vs. what it fetches via tools.
6. **Set guardrails** — especially: does it *send* communications, or *draft* them for your approval?
7. **Build the loop** — wire model + tools + context together.
8. **Test on real examples** — run it against actual past handoffs, see where it breaks, refine.

The good news: you don't have to build the loop, permission system, or tool-execution machinery from scratch. Claude Code and the Claude Agent SDK give you those for free — your real design work is steps 1–6.

Clarifying questions

1. **The task itself.** Walk me through what design handoff looks like *manually* today.
2. **Where do the inputs live?** Figma? A design system? Specs in docs or tickets?
3. **Where do the communications go?** Email? Teams? A ticketing system? Who's the audience?
4. **Draft vs. send.** Should the agent *draft* for review, or *send* autonomously?
5. **What does "build" mean at the hackathon?** Working prototype, or designed-and-documented harness?
6. **Your comfort zone.** Comfortable with code, or prefer configuration-and-prompts?

BRIAN

1. Today, this means that the working group attached to the design project (designer(s), PM, dev) agree that the design is complete. Historically, we would then prepare the file for handoff by annotating the designs, highlighting important details, and filling in any gaps that were identified. Engineering then would be given access to the file, and we would write a ticket for them describing the task and including a link to the Figma file. If the project had required novel UI, not covered by the existing design system, a new DS component would have to be defined and created, which would be kind of an offshoot from the main effort. Today, we are almost always delivering prototypes created with Claude Code, hosted on Vercel. Dev goes to the hosted site to get familiar with the new feature and inspect value. It's still a translation. Then engineering does their thing, building the feature out in one sprint or another. Once completed in dev, the feature is promoted through our SQA cycle from dev to QA, from QA to test, and from test to production. Sometimes engineers come to us for clarification as they're building. Sometimes we're brought in to review the test build, but rarely. More often, we don't hear anything for weeks or months, until the feature has been released and people are encountering issues with it. As the feature is moving through the quality assurance phases product prepares release notes, and, sometimes, packs of information describing the new feature for internal audiences (support, sales, other product teams, execs).

2. The inputs are the Figma file or Claude Code prototype, our design system (if applicable), and the existing codebase. Specs have been written in Figma in the past but, increasingly, we are generating them from the final prototype. These details are sometimes consumed directly, sometimes added to Jira tickets or Excel spreadsheets. Figma and Microsoft are definitely in play, though I haven't gotten the MS365 connector to work (not authorized by my company yet).

3. Right now the communications are informal and unstructured, if they exist at all. It's ad-hoc. Sometimes we're on the ball and communicate impending changes to our support teams, account managers, and sales people, but more often they're surprised by something new they see in the product one day and we get angry chats and emails. That's one of the things we want to fix with

this effort. If the process of disseminating upcoming UX changes were more automatic, and less manual, the thinking is we could make a habit of it. There probably need to be different comms for different audiences. The design system team gets one message. The support folks and technical account managers get another, explaining how the feature works, what changed, how to use and how not to use it etc. QA gets instructions for testing it. Dev gets the new feature as spec'd UI code ready for import, maybe with accompanying documentation similar to the support content. Release notes get drafted, briefly stating the most important aspects of the change. A middle-weight description of the change is authored for internal presentations or blogs.

4. I imagine it will send things, but perhaps with a human in the loop to review and edit the content.

5. A working prototype if we can swing it.

6. I'm comfortable with coding, so long as you're writing it. I can read it somewhat, but I don't have much technical expertise.

CLAUDE

This is a rich picture, and honestly a textbook-perfect agent use case. Let me reflect back the shape of what you're building, then lock a few decisions with you.

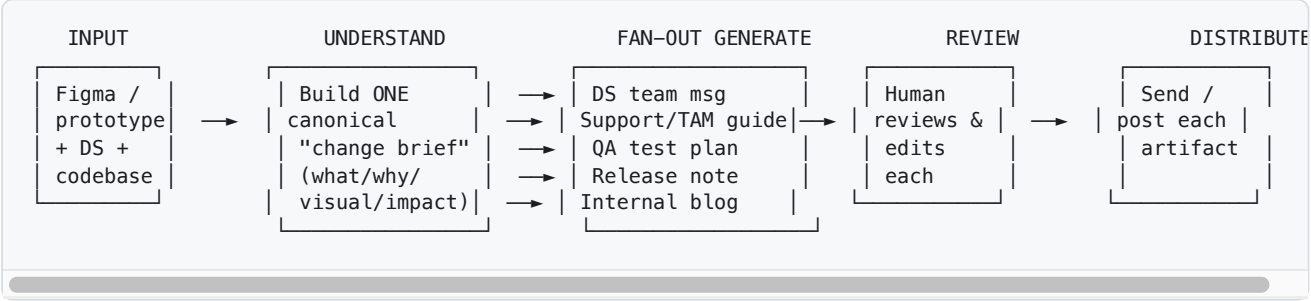
The shape of your agent

Buried in everything you described is one crisp, high-value job the agent can own:

> *Take a completed design change and fan it out into the right message for every downstream audience — reviewed by a human before it goes out.*

That "surprise → angry chat" problem you named is the real prize. The dev handoff is *already* half-solved by Claude Code + your hosted prototypes. The **multi-audience communications** part is where the pain is, where it's currently ad-hoc, and where an agent shines — because it's the same underlying change retold six different ways.

Here's the loop:



The key architectural idea — and the thing that'll make it *good* — is that middle step. The agent first distills the change into **one structured "change brief"** (what changed, why, before/after visuals, which DS components are affected, is a new component needed). Then each audience artifact is generated

from that same brief by a specialized generator. One understanding, many tailored retellings. That's also naturally a place to fan out into parallel subagents, one per audience.

Mapped to the six harness pieces (your version)

Piece	Your build
Loop	Understand → fan-out generate → human review → distribute
Instructions	Per-audience "voice + what they care about" specs (QA wants edge cases; sales wants benefits; DS wants component impact)
Context	The change brief + design system reference + the prototype/Figma. Fetched via tools, not all crammed in up front
Tools	Read Figma, fetch the hosted prototype, read DS reference, read codebase, and the *distribution* tools (email/Teams/Jira)
Guardrails	Nothing sends without human approval — the review gate is the whole safety story for a comms agent
Evaluation	Run it on 2–3 *past* real changes; compare its artifacts to what you'd have written by hand

The one real blocker, and what to prep now

MS365 isn't authorized for you. That's your critical-path risk, because "distribute" depends on it.

1. **Start the authorization request now** — it's the longest lead-time item and it's not in your control. Plan a fallback so a demo isn't hostage to IT.
2. **Gather 2–3 real past changes** with their eventual outcomes. These become your test cases *and* your demo material.

Everything else — Figma reading, the change brief, the fan-out generation, a review UI — you can build regardless of the MS365 situation.

Three decisions locked with the user:

BUILD SCOPE

How wide should the hackathon build reach? ✓ Full pipeline

Understand → dev handoff → comms → distribution, end to end. Most complete story, but ambitious for two days.

HARNESS STACK

What should the harness run on? ✓ Agent SDK app on Vercel

A small TypeScript app using the Claude Agent SDK plus a minimal review/approve UI, deployed to Vercel.

DISTRIBUTION

How should the agent "send" for the build? ✓ Review UI, defer sending

Agent produces artifacts into a review/approve UI; actual sending stubbed or manual. Sidesteps the MS365 blocker.

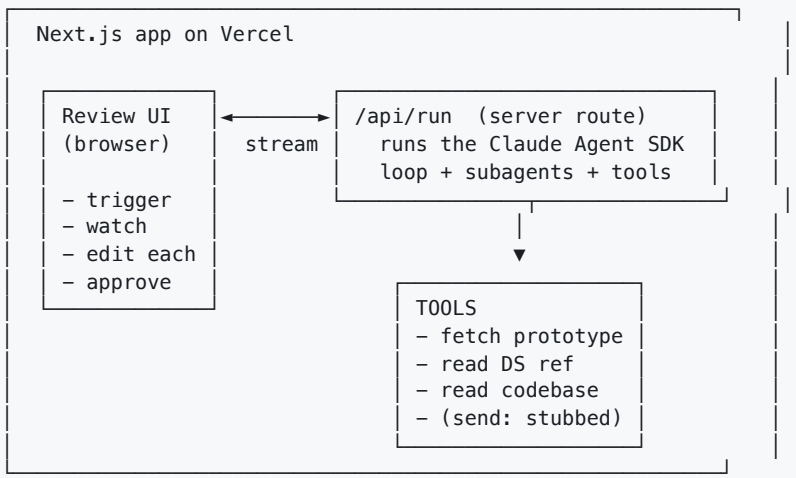
CLAUDE

Good — you went ambitious on scope, pragmatic on the rest. That's a workable combination *if* we build it the right way.

The one honest caveat

"Full pipeline" in 2 days is a lot. The way you don't crash into that wall is **spine-first**: build a thin version that runs end-to-end (one input → brief → *one* artifact → review UI) on day one, then widen it stage by stage. That way you always have something that works to demo.

The architecture (Agent SDK app on Vercel)



- **The agent** = Claude Agent SDK (TypeScript). Same harness machinery as Claude Code — loop, tools, subagents, permissions — but *yours*, wired to your task.
- **The UI** = a Next.js page: trigger a run, watch progress stream in, then a card per audience you can edit and approve.
- **Deploy** = Vercel, which you already use. (Function timeout is now 300s, plenty for bounded stages; we stream progress so it never feels stuck.)

Input strategy — lean on the prototype, not Figma

The most important simplification: **make the hosted prototype URL + its source code the primary input, and Figma optional.**

1. You're increasingly generating specs *from the prototype* anyway.
2. Figma's MCP runs locally against your Figma app — awkward to reach from a deployed Vercel server. The prototype is just a URL to fetch and a repo to read.

So the agent's front door becomes: **a prototype URL + a short human note** ("what changed and why"). Everything else it derives.

The full pipeline, staged

Stage	Input → Output	Tools	Spine phase
1. Understand	prototype + note → **change brief** (what/why/before-after/affected DS components/new-component?)	fetch prototype, read DS ref, read codebase	✓ Phase 0
2. Dev handoff	brief → spec'd UI notes + import-ready code pointer + dev doc	read codebase	Phase 2
3. Comms fan-out	brief → 5 audience artifacts (DS / support+TAM / QA / release note / internal blog), generated in parallel	—	✓ start with 1, widen
4. Review	artifacts → editable cards, approve each	—	✓ Phase 0
5. Distribute	approved → send (stubbed: download / copy / "mark sent")	send-tool (stub now, MS365 later)	✓ stub in Phase 0

Phased build

- **Phase 0 (spine):** prototype URL + note → change brief → *release note only* → review card → stub "send." End to end. Day-1 morning target.
- **Phase 1:** widen stage 3 to all five audiences, fanned out as parallel subagents (one generator per audience, each with its own "voice + what they care about" instructions).
- **Phase 2:** add the dev-handoff stage and the "is a new DS component needed?" detection.
- **Phase 3 (if time):** real distribution via whatever channel landed.

What to do in the weeks before the build

1. **File the MS365 authorization request now** — longest lead time, not in your control.
2. **Collect 2–3 real past changes** — the prototype URL + what release note/support content actually got written + any complaint that came in.
3. **Write the audience specs** — for each of the five audiences: who they are, what they care about, tone, and one example of a *good* artifact. This is 80% of what makes the output good.
4. **Confirm the five audiences and their channels.**

What I need to start scaffolding

- **One sample prototype URL** (a real hosted feature) to wire the "understand" stage against.
- **Confirmation of the five audiences** (or your edits to that list).
- A yes on the stack: **Next.js + Claude Agent SDK (TypeScript) on Vercel.**

Recommended next move: work through the **audience specs** first, because a great harness with vague instructions produces mediocre output — nailing those makes the build days mostly wiring.